**University of Tehran**

College of Engineering

School of Electrical and Computer Engineering

Computer Assignment Report

# Hardware Accelerator Design for Sobel Filter

FPGA-based Embedded Systems Design

Analysis and Comparison of Baseline, 1-Cycle Streaming, and 3-Stage Pipelined Architectures on Zynq-7020 SoC with SVA Formal Verification

**Mohammad Khoshroo**

Student ID: 810102441

Dr. Bijan Alizadeh

Fall 2026

# Contents


1- Introduction....1
2- Hardware Architecture and Circuit Design....1
2-1- Baseline Architecture....1
A) Baseline Datapath....1
B) Baseline Controller FSM....3
2-2- Optimized 1-Cycle Streaming Architecture....4
2-3- 3-Stage Pipelined Architecture....7
3- Simulation....7
4- Verification with SystemVerilog Assertions (SVA)....12
4-1- Types of Covered Verification Scenarios....12
1. Reset State Verification....12
2. Output Validity and Unknown State Prevention....12
3. Exact Datapath Latency Verification....12
4. Dynamic Range and Saturation Clamping....12
5. Control Protocol Consistency and Mutual Exclusion....13
4-2 Bonus: Intentional Bug Injection Scenarios....14
5- Vivado Synthesis and Implementation Results....15
5-1- Unconstrained Implementation Analysis....15
A) Consequences of Omitting Clock Constraints and Absence of BRAM Inference....15
B) Intermediate Stage: Memory Rewriting and Block RAM Inference (BRAM Inference)....15
5-2- Constrained Implementation and Timing Optimization....15
Table 5-1: Analytical Comparison Across the Three Implementation Stages of the Baseline Architecture....17
Table 5-2: Hardware Resource Utilization in Final Constrained Mode....17
5-4- Timing Analysis and Maximum Frequency ($F_{\max}$)....18
Mathematical Formulation of Maximum Operating Frequency ($F_{\max}$)....18
5-5- Critical Path Analysis....18
5-6- Vivado Implementation Screenshots....19
1. Initial Unconstrained Implementation and 176W Power Anomaly (Files 01.png, 02.png, 03.png)....19
2. Intermediate Stage: BRAM Inference Without Clock Constraints (Files with ‘true’ Suffix)....20
3. Final Constrained Implementation at 100 MHz....21
4. Implementation Results for Optimized Streaming Architecture....24
5. Implementation Results and Device Floorplan for Pipelined Architecture....26

# 1- Introduction

The Sobel edge detection algorithm utilizes two $3 \times 3$ convolution kernels to compute the spatial gradient of image luminance along the horizontal axis ($G_x$) and vertical axis ($G_y$):

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * I$$

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * I$$

In this project, the total gradient magnitude is obtained using the standard Manhattan distance approximation (sum of absolute values of orthogonal components), defined as:

$$G = |G_x| + |G_y|$$

To prevent arithmetic overflow during intermediate accumulations, the internal data width must be maintained with sufficient precision (11 bits). In the final stage, the gradient magnitude is saturated (clamped) to the standard 8-bit dynamic range ($[0, 255]$):

$$G_{\text{sat}} = \begin{cases} 255 & \text{if } G > 255 \\ G[7:0] & \text{if } G \leq 255 \end{cases}$$

> The input test frame has dimensions of $640 \times 853$ pixels (totalling $545,920$ pixels). Because the $3 \times 3$ convolution operator requires a 1-pixel border around the active frame, the valid output region spans $638 \times 851$ pixels (totalling $542,938$ valid active pixels).

# 2- Hardware Architecture and Circuit Design

This chapter details the design, microarchitecture, datapath structures, and finite state machine (FSM) controllers for three progressive hardware implementations: the area-efficient Baseline architecture, the high-throughput 1-Cycle Streaming architecture, and the 3-Stage Pipelined architecture.

---

## 2-1- Baseline Architecture

The baseline architecture was designed with an area-minimization objective, employing time-multiplexing of arithmetic operators across multiple clock cycles.

### A) Baseline Datapath

In this architecture, a shared arithmetic unit computes both $G_x$ and $G_y$ gradient components. Based on the `xy_ctrl` control signal, the active window pixels are routed to a Carry-Save Adder (CSA) tree. The datapath consists of the following key components:

- **4-Stage CSA Tree:** Combines the 6 active pixel operands of the Sobel window into 11-bit sum and carry vectors.
- **XM and YM Multiplexers:** These 4-to-1 multiplexers route the CSA sum and carry vectors, inverted feedback register values (for two's complement subtraction), and constant zero to the inputs of the final Carry-Propagate Adder (CPA).
- **Feedback Registers (`reg_Gx` and `reg_Gy`):** Store intermediate gradient components across consecutive computation cycles.
- **Per-Pixel Latency:** Processing each pixel requires 4 consecutive clock cycles ($G_x$ calculation, two's complement negation if negative, $G_y$ calculation, and absolute summation with dynamic range saturation clamping).

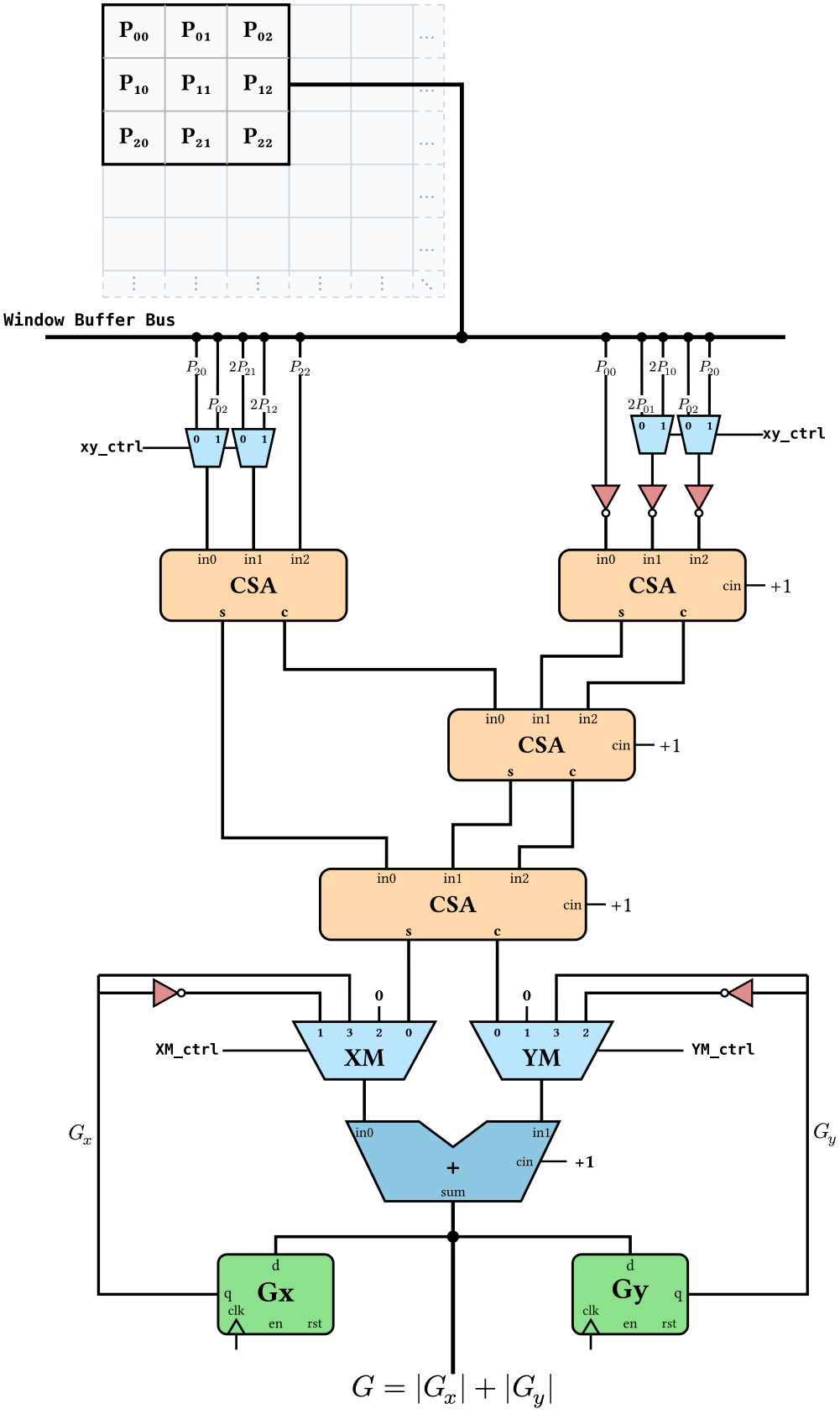



Figure 1: Complete Datapath Schematic of the Baseline Architecture

## B) Baseline Controller FSM

The accelerator controller executes the following sequence of states:

1. `IDLE` : Awaits assertion of the `start` trigger and resets internal counters.
2. `LOAD_CHECK` : Verifies whether initial line buffers are sufficiently filled.
3. `LOAD_ROW` : Writes new row data into line buffers and shifts row pointers.
4. `PREPARE_WIN` : Loads the first two columns of the $3 \times 3$ sliding window.
5. `CALC_GX` : Computes the horizontal gradient component by activating $X$-control signals.
6. `NEG_GX` : If the sign bit $G_x[10]$ is negative, performs two's complement negation via the feedback register.
7. `CALC_GY` : Computes the vertical gradient component by activating $Y$-control signals.
8. `NEG_GY` : Performs two's complement negation if the sign bit $G_y[10]$ is negative.
9. `DONE_PIXEL` : Asserts the `valid_pixel` strobe, applies saturation clamping, and evaluates end-of-row and end-of-frame conditions.
10. `DONE` : Asserts the `done` completion interrupt pulse.

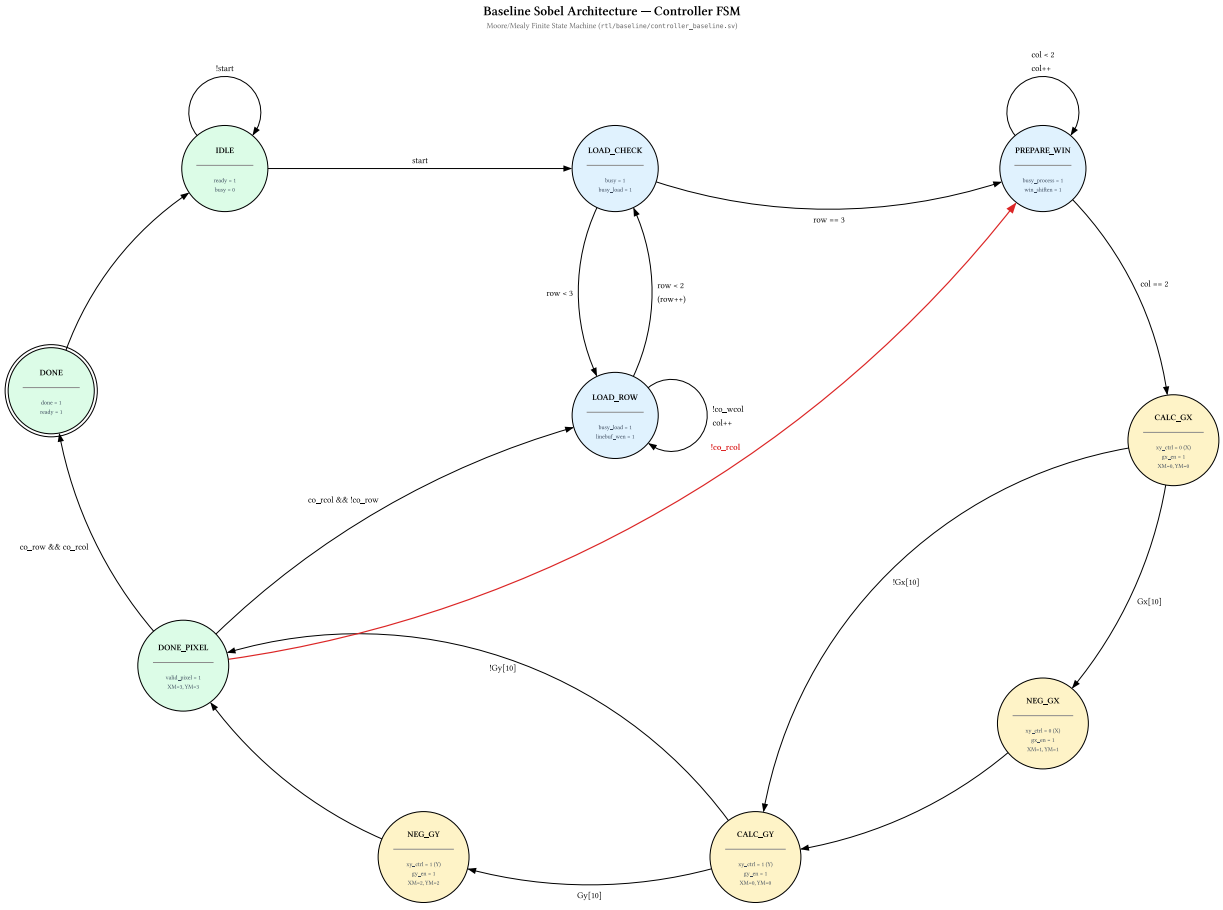



Figure 2: State Transition Diagram of the Baseline Controller FSM

## 2-2- Optimized 1-Cycle Streaming Architecture

While the baseline architecture achieves minimal logic area, its requirement of 4 to 5 clock cycles per pixel severely constrains throughput ($3,040,078$ cycles per frame).

To achieve single-cycle throughput, the column sum for column $k$ ($k \in \{0,1,2\}$) of the sliding window is defined as:

$$c_k = P_{0,k} + 2 \cdot P_{1,k} + P_{2,k}$$

As the window shifts horizontally across the image, the current right column ($c_2$) becomes the center column ($c_1$) in the next clock cycle, and subsequently becomes the left column ($c_0$) in the following cycle. Consequently, the circuit does not need to recompute all 6 pixels each cycle. Instead, only the newly arrived rightmost column ($c_2$) is computed and shifted through a 2-stage delay line:

$$c_2 \overset{\text{clk}}{\rightarrow} c_1 \overset{\text{clk}}{\rightarrow} c_0$$

The horizontal gradient is then obtained by a direct subtraction of the outer column sums:

$$G_x = c_2 - c_0 = c_2 + \overline{c_0} + 1$$

In the baseline architecture, computing absolute values required sign evaluation followed by an additional negation cycle. In the optimized streaming architecture, $|G_x|$ and $|G_y|$ are computed combinationaly in parallel using XOR bitwise negation driven by the sign bit ($S = G[10]$):

$$|G| = (G \oplus \{11\{S\}\}) + S$$

This value is integrated directly into the final adder network, providing the saturated output in the very same clock cycle.

Furthermore, line buffer memory requirements are reduced from 3 image rows to only 2 rows. The incoming streaming pixel is applied directly as the bottom row ($P_{2,k}$) of the convolution window while simultaneously being stored in on-chip Block RAM. As a result, BRAM utilization is reduced from 3 RAMB18 blocks (1.5 tiles) to **2 RAMB18 blocks (1.0 tile)** — a 33.3% reduction in on-chip memory footprint.

The optimized controller operates across 4 balanced states:

- `IDLE` : Reset and idle state awaiting execution start.
- `STREAM` : Continuous streaming state; line buffers and sliding window registers shift concurrently with each incoming pixel. Following the initial line-buffer fill latency, `valid_pixel` remains continuously asserted high.
- `FLUSH` : Pipeline draining and window flushing to complete edge boundary pixels at row boundaries.
- `DONE` : Asserting the `done` interrupt signal at cycle $545,926$.

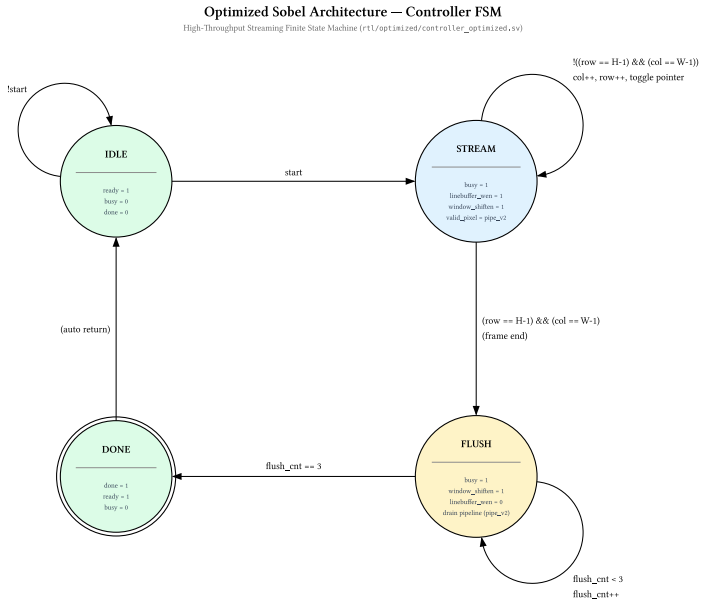



Figure 3: State Transition Diagram of the Optimized Controller FSM

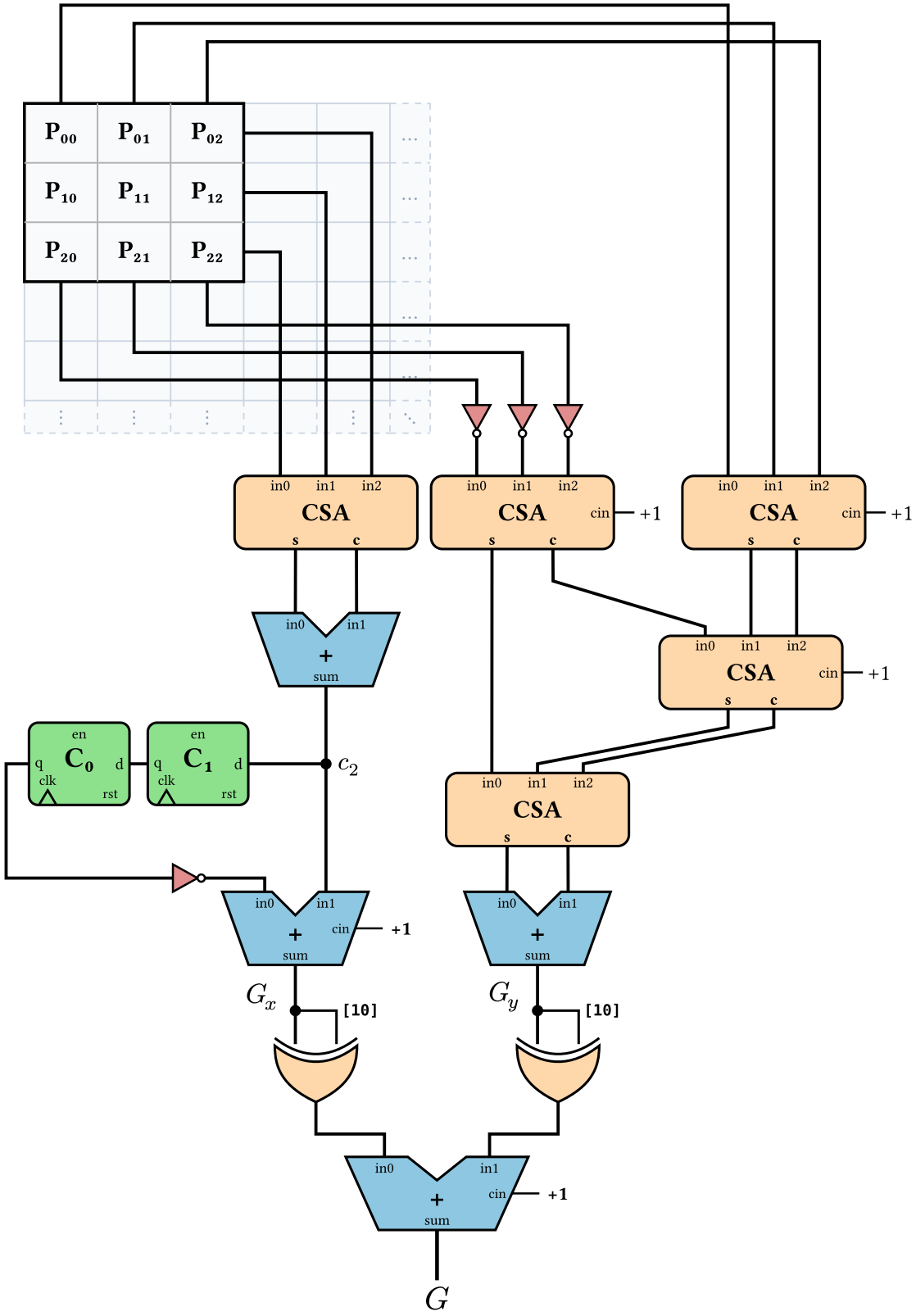



Figure 4: Complete Hardware Circuit Schematic of the Optimized Streaming Architecture

### 2-3- 3-Stage Pipelined Architecture

To maximize clock frequency and break the long combinational path spanning the CSA tree to the final saturation adder, a 3-stage pipelined architecture was designed:

- **Stage 1 (Fetch & Column Sum):** Fetches incoming pixels, computes the rightmost column sum $c_2$ using a Carry-Save Adder, and computes positive and negative vertical terms ($G_y$). Results are registered into Stage 1 pipeline registers.
- **Stage 2 (Gradient Accumulation):** Completes signed horizontal ($G_x$) and vertical ($G_y$) gradient evaluations, and applies parallel XOR inversion gates for absolute-value preparation. Results are latched into Stage 2 pipeline registers.
- **Stage 3 (Magnitude & Saturation):** Sums absolute gradient components, applies dynamic range saturation clamping to $[0, 255]$, and asserts `pixel_out` alongside `valid_pixel`.

---

## 3- Simulation

Following RTL simulation and output generation (`image_out.hex`), the Python script `Checker.py` was executed to quantitatively compare the hardware accelerator outputs against a double-precision OpenCV software reference model.



**Input Grayscale Image (640 × 853)**



**Hardware Sobel Output**

Figure 5: Sample 1 Comparison

To comprehensively validate hardware accuracy, a **benchmark test suite consisting of 5 diverse images** was simulated across all **three hardware architectures** (Baseline, Optimized Streaming, and 3-Stage Pipelined), totalling 15 complete simulation runs with full RTL assertion checks. Each

output was evaluated both across the full frame (incorporating zero-padding border semantics per `Checker.py`) and strictly within the active $638 \times 851$ valid filtering region.

The numerical evaluation results are categorized per input image in the tables below:

**Table 3-1: Error Evaluation for Input Image 1 (input1)**

| Hardware Architecture | Full-Frame Mean Error (Checker.py) | Full-Frame Max Error | Active Region Mean Error | Active Region Max Error |
|---|---|---|---|---|
| **Baseline** | **0.10** (0.1046) | 255 | **0.0000** | 0 |
| **Optimized** | **0.22** (0.2196) | 255 | **0.1156** | 52 |
| **Pipeline** | **0.22** (0.2196) | 255 | **0.1156** | 52 |

**Table 3-2: Error Evaluation for Input Image 2 (input2)**

| Hardware Architecture | Full-Frame Mean Error (Checker.py) | Full-Frame Max Error | Active Region Mean Error | Active Region Max Error |
|---|---|---|---|---|
| **Baseline** | **0.23** (0.2296) | 255 | **0.0000** | 0 |
| **Optimized** | **0.35** (0.3476) | 255 | **0.1186** | 167 |
| **Pipeline** | **0.35** (0.3476) | 255 | **0.1186** | 167 |

**Table 3-3: Error Evaluation for Input Image 3 (input3)**

| Hardware Architecture | Full-Frame Mean Error (Checker.py) | Full-Frame Max Error | Active Region Mean Error | Active Region Max Error |
|---|---|---|---|---|
| **Baseline** | **0.02** (0.0163) | 255 | **0.0000** | 0 |
| **Optimized** | **0.02** (0.0164) | 255 | **0.0001** | 6 |
| **Pipeline** | **0.02** (0.0164) | 255 | **0.0001** | 6 |

**Table 3-4: Error Evaluation for Input Image 4 (input4)**

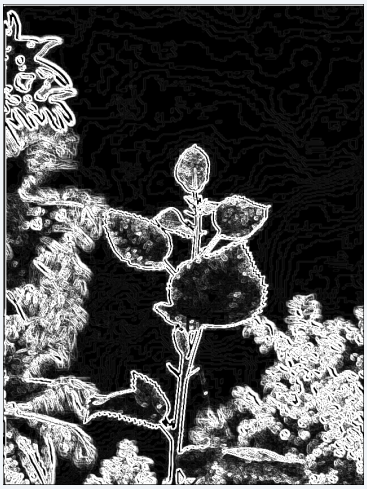
| Hardware Architecture | Full-Frame Mean Error (Checker.py) | Full-Frame Max Error | Active Region Mean Error | Active Region Max Error |
|---|---|---|---|---|
| **Baseline** | **0.00** (0.0000) | 0 | **0.0000** | 0 |
| **Optimized** | **0.00** (0.0020) | 1 | **0.0020** | 1 |
| **Pipeline** | **0.00** (0.0020) | 1 | **0.0020** | 1 |

**Table 3-5: Error Evaluation for Input Image 5 (input5)**

| Hardware Architecture | Full-Frame Mean Error (Checker.py) | Full-Frame Max Error | Active Region Mean Error | Active Region Max Error |
|---|---|---|---|---|
| **Baseline** | **0.01** (0.0144) | 255 | **0.0000** | 0 |
| **Optimized** | **0.02** (0.0223) | 255 | **0.0079** | 56 |
| **Pipeline** | **0.02** (0.0223) | 255 | **0.0079** | 56 |

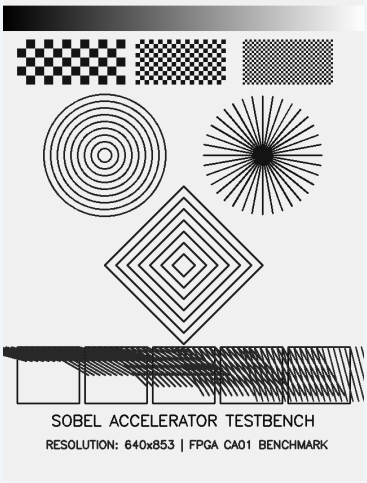


**Input Grayscale Image (640 × 853)**

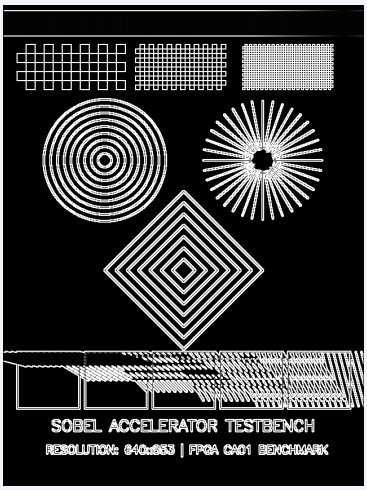


**Hardware Sobel Output**

Figure 6: Sample 2: Tehran University Gate

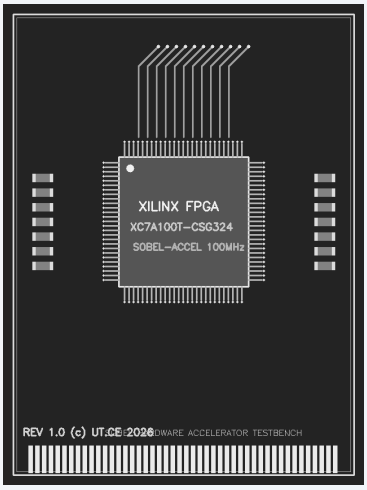



**Input Grayscale Image (640 × 853)**

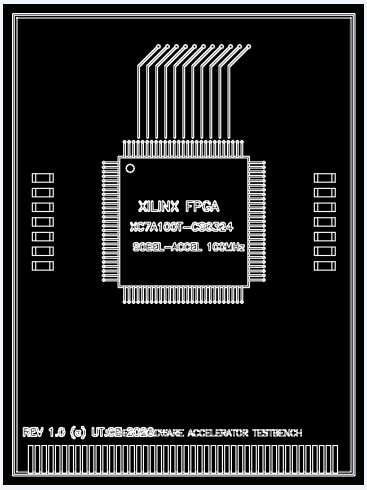



**Hardware Sobel Output**

Figure 7: Sample 3: Geometric & Frequency Pattern

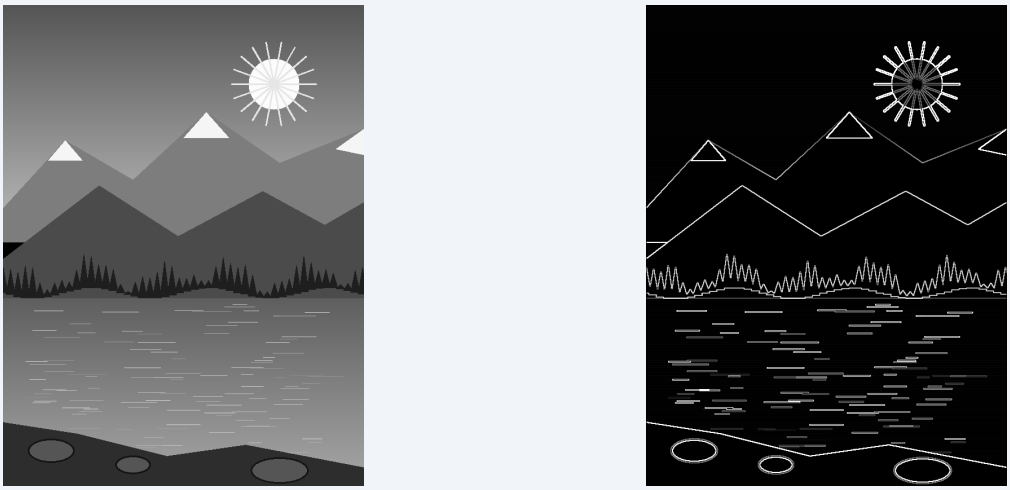



**Input Grayscale Image (640 × 853)**




**Hardware Sobel Output**

Figure 8: Sample 4: FPGA Circuit Board



**Input Grayscale Image (640 × 853)** **Hardware Sobel Output**

Figure 9: Sample 5: Mountain Natural Landscape

| Hardware Architecture | Clock Cycles / Frame | Total Latency @ 100MHz | Pixel Throughput | Frame Rate (FPS) | Speedup Factor |
|---|---|---|---|---|---|
| **Baseline** | 3,040,078 | 30.401 ms | 17.86 Mpx/s | 32.89 | 1.00x (Ref) |
| **Optimized** | **545,926** | **5.459 ms** | **99.45 Mpx/s** | **183.18** | **5.57x** |
| **Pipeline** | **545,929** | **5.459 ms** | **99.45 Mpx/s** | **183.17** | **5.57x** |

The baseline architecture required 4 to 5 cycles per pixel, completing frame processing in $3,040,078$ cycles. In contrast, the optimized streaming architecture begins output production at cycle $1,284$ and produces exactly one pixel per clock cycle thereafter, completing the entire frame in only $545,926$ cycles — representing a **5.57x speedup**.

At the standard 100 MHz operating frequency, the system achieves **183.18 frames per second**. When operated at the chip's maximum allowable frequency ($F_{\text{max}} = 212.86$ MHz), throughput scales beyond **380 FPS**, fully supporting ultra-high-speed real-time video processing.

# 4- Verification with SystemVerilog Assertions (SVA)

For rigorous formal and dynamic protocol verification, a dedicated `sva/` directory was established containing checker modules coupled to each core module. Immediate assertions (`assert`) were utilized for instantaneous combinational validity checks, while concurrent temporal assertions (`assert property`) were formulated to verify multi-cycle protocols, pipeline delays, and state invariants.

## 4-1- Types of Covered Verification Scenarios

### 1. Reset State Verification

Verifies that immediately upon assertion of asynchronous reset, all control outputs are forced to their inactive state and `ready` asserts high:

```systemverilog
CTRL_SIG_RESET: assert property (@(posedge clk) rst |->
    (!valid_pixel && !busy && !done && ready))
else $error("[SVA] Control signals not in IDLE state during reset!");
```

### 2. Output Validity and Unknown State Prevention

Guarantees that `pixel_out` is evaluated only when `valid_pixel` is asserted, and verifies that no control signal takes an unknown ($X$ or $Z$) value during execution:

```systemverilog
VALID_PIXEL_KNOWN: assert property (@(posedge clk) disable iff (rst)
    valid_pixel |-> !$isunknown(pixel_out))
else $error("[SVA] valid_pixel is high but pixel_out is X or Z!");

VALID_IMPLIES_BUSY: assert property (@(posedge clk) disable iff (rst)
    valid_pixel |-> busy)
else $error("[SVA] valid_pixel asserted while controller is not busy!");
```

### 3. Exact Datapath Latency Verification

In the optimized streaming architecture, producing the first valid output pixel requires buffering the initial two image rows ($2 \times 640$ pixels) plus 4 startup cycles for the sliding window pipeline. The following temporal assertion verifies that `valid_pixel` asserts at precisely cycle $1,284$:

```systemverilog
localparam int EXPECTED_LATENCY = (2 * WIDTH_SIZE) + 4; // 1284 cycles

CORRECT_VALID_ISSUE: assert property (@(posedge clk) disable iff (rst)
    (ready && start) |-> ##(EXPECTED_LATENCY + 1) valid_pixel)
else $error("[SVA] valid_pixel did not assert at the expected cycle!");
```

### 4. Dynamic Range and Saturation Clamping

Ensures that output values strictly conform to the 8-bit dynamic range $[0, 255]$, and verifies that any computed gradient magnitude exceeding 255 is clamped to 255:

```systemverilog
G_SAT_BOUND: assert property (@(posedge clk) disable iff (rst)
    int'(G_sat) >= 0 && int'(G_sat) <= 255)
else $error("[SVA] G_sat output exceeds 8-bit bounds [0, 255]!");

SATURATION_CLAMP: assert property (@(posedge clk) disable iff (rst)
    (G < 11'd256) || (G_sat == 8'd255))
else $error("[SVA] Saturation failure! Gradient G=%0d exceeded 255 but G_sat was not clamped to 255 (got %0d)", G, G_sat);
```

### 5. Control Protocol Consistency and Mutual Exclusion

Verifies protocol handshaking invariants, including mutual exclusion between `ready` and `busy`, proper transition to the active state upon receiving `start`, and return to `ready` upon completion:

```systemverilog
MUTUAL_EXCL: assert property (@(posedge clk) disable iff (rst)
    !(ready && busy))
else $error("[SVA] Controller cannot be both ready and busy!");

START_TO_STREAM: assert property (@(posedge clk) disable iff (rst)
    (ready && start) |=> (busy && !ready))
else $error("[SVA] start asserted while ready did not transition to busy!");

DONE_TO_IDLE: assert property (@(posedge clk) disable iff (rst)
    done |=> (ready && !busy))
else $error("[SVA] done asserted did not return to ready next cycle!");
```

### Golden Simulation Run Results

A complete simulation run was executed on the $545,920$-pixel frame with all SVA assertions active. The simulation log extracted from `sim/assertion_bug_logs/golden_clean.log` confirms that zero assertion violations occurred:

```Text
======================================================================
   Sobel Optimized Accelerator Testbench with SVA Verification
   Image Dimensions: 640 x 853 | Total Pixels: 545920
======================================================================
[INFO] Loading input image hex from: inputs/input1/image_in.hex ...
[INFO] Successfully loaded 545920 input pixels.
[Time 12905000 ns] First valid pixel detected!
[Time 12915000 ns] [OPTIMIZED] Valid Pixel #1 = 0x00 (0)
[SVA controller_optimized] Measured latency from start to first valid_pixel: 1284 cycles (Expected: 1284)
...
[Time 5459305000 ns] DUT asserted DONE interrupt!
----------------------------------------------------------------------
Optimized Simulation Complete:
  Total Input Pixels Fed     : 545920 / 545920
  Total Valid Pixels Produced: 542938 / 542938
----------------------------------------------------------------------
Saved optimized hex output to sim/image_out_optimized.hex
- S i m u l a t i o n   R e p o r t: Verilator 5.050 / QuestaSim
- Status: PASSED (Zero Assertion Violations)
```

## 4-2 Bonus: Intentional Bug Injection Scenarios

To empirically demonstrate assertion coverage and fault sensitivity, five intentional hardware defects were inserted into copies of the controller and datapath modules in `rtl/bugged_optimized/`. Each scenario was simulated independently. As summarized below, the SVA assertions immediately flagged the fault at the exact cycle of occurrence and terminated the simulation:

| Bug ID | Injected Fault Scenario | Modified Module | Error Message Extracted from Simulation Log ( `sim/assertion_bug_logs/` ) |
|---|---|---|---|
| **Bug 1** | Control signals remaining active during reset | `controller_optimized.sv` | `%Error: controller_optimized_sva.sv:74: Assertion failed in a_reset_state: [SVA controller_optimized] Signals not idle during reset!` |
| **Bug 2** | Tampering with window startup delay counter | `controller_optimized.sv` | `%Error: controller_optimized_sva.sv:106: Assertion failed in a_datapath_latency: [SVA controller_optimized] Datapath latency mismatch! Expected 1284 cycles, got 644` |
| **Bug 3** | Failure to transition to Busy state upon Start pulse | `controller_optimized.sv` | `%Error: controller_optimized_sva.sv:82: Assertion failed in a_start_to_busy: [SVA controller_optimized] start asserted while ready did not transition to busy!` |
| **Bug 4** | Premature valid_pixel assertion before buffer filling | `controller_optimized.sv` | `%Error: controller_optimized_sva.sv:106: Assertion failed in a_datapath_latency: [SVA controller_optimized] Datapath latency mismatch! Expected 1284 cycles, got 0` |
| **Bug 5** | Removal of saturation clamping logic | `core_optimized.sv` | `%Error: core_optimized_sva.sv:31: Assertion failed in SATURATION_CLAMP: [SVA core_optimized] Saturation failure! Gradient G=450 exceeded 255 but G_sat was not clamped to 255 (got 194)` |

# 5- Vivado Synthesis and Implementation Results

All SystemVerilog design files were synthesized and physically implemented targeting the **Xilinx Zynq-7020 SoC (`xc7z020clg400-1`)** using **Vivado 2022.2**.

## 5-1- Unconstrained Implementation Analysis

Implementation in Vivado was carried out in two fundamental stages: initially performing physical implementation **prior to defining clock constraints** to observe inherent circuit behavior and synthesis warnings, followed by **applying a 100 MHz clock constraint via XDC** for accurate timing and power optimization.

### A) Consequences of Omitting Clock Constraints and Absence of BRAM Inference

1. **Undefined Timing Status:** In the absence of a user clock constraint, Vivado assumes no specific operating frequency. The following warning appeared in the timing summary:

```
There are no user specified timing constraints.
```

Consequently, WNS, TNS, and timing slack were reported as `NA` or `inf`, preventing the tool from validating setup or hold stability.

2. **Power Dissipation Explosion (176.174 W) and Thermal Violation:** Vivado's power analysis engine assumes aggressive default switching activity when unconstrained. As a result, total estimated on-chip power surged to an alarming **176.174 W**, triggering the critical thermal violation:

```
Junction temp exceeded!
```

3. **Line Buffer Synthesized on Distributed Logic (LUT-RAM / SRL):** Due to the initial coding style of the memory array, the line buffers failed to infer dedicated hardware Block RAM primitives and were mapped as distributed shift registers across FPGA logic slices. This alone caused an exorbitant utilization of **6,402 Slice LUTs** (12.03% of device capacity) and **15,511 Slice Registers** (14.58%).

### B) Intermediate Stage: Memory Rewriting and Block RAM Inference (BRAM Inference)

Prior to applying clock constraints, the line buffer HDL descriptions were restructured to allow Vivado to automatically infer dedicated `RAMB18E1` primitives:

- Logic resource utilization plummeted dramatically: Slice LUT usage dropped from 6,402 to **149 (0.28%)**, flip-flops dropped from 15,511 to **127 (0.12%)**, and buffers were accommodated inside **3 RAMB18E1 blocks (1.5 tiles)**.
- Total unconstrained estimated power dropped from 176.174 W to **8.100 W** (>95% power reduction).

## 5-2- Constrained Implementation and Timing Optimization

To evaluate timing and circuit stability at 100 MHz (10.0 ns clock period), a timing constraint was specified in `pin_assignment.xdc`:

```tcl
create_clock -period 10.000 -name CLK_100MHz [get_ports clk]
```

Upon reapplying synthesis and implementation with this constraint:

1. All intra-clock paths were analyzed, yielding a setup slack of **WNS = +3.299 ns** with zero timing violations (TNS = 0.00 ns).

2. Realistic power estimation based on 100 MHz operation reduced total on-chip power to an ultra-low **0.117 W (117 mW)**.
3. At an ambient temperature of 25.0°C, the junction temperature stabilized at a cool **26.3°C**, providing a generous thermal margin of **+58.7°C**.

**Table 5-1: Analytical Comparison Across the Three Implementation Stages of the Baseline Architecture**

| Metric / Parameter | Stage 1: Initial (Unconstrained, Distributed Logic) | Stage 2: BRAM Inferred (Unconstrained, BRAM) | Stage 3: Final Constrained (@ 100MHz + BRAM) |
|---|---|---|---|
| **Worst Negative Slack (WNS)** | NA (undefined / inf) | NA (undefined / inf) | **+3.299 ns** |
| **Total Power** | **176.174 W** | **8.100 W** | **0.117 W (117 mW)** |
| **Dynamic Power** | 175.135 W (99.4%) | 7.263 W (89.7%) | **0.012 W (12 mW)** |
| **Device Static Power** | 1.039 W | 0.837 W | **0.105 W (105 mW)** |
| **Junction Temperature** | **125.0 °C** | **118.4 °C** | **26.3 °C** |
| **Thermal Margin** | **-1971.8 °C** | **-33.4 °C** | **+58.7 °C** |
| **Slice LUTs** | **6,402** (12.03%) | **149** (0.28%) | **148** (0.28%) |
| **Slice Registers (FF)** | **15,511** (14.58%) | **127** (0.12%) | **127** (0.12%) |
| **Block RAM (RAMB18E1)** | **0** | **3** (1.5 tiles - 1.07%) | **3** (1.5 tiles - 1.07%) |

**Table 5-2: Hardware Resource Utilization in Final Constrained Mode**

| Hardware Resource | Zynq-7020 Available | Baseline Architecture | Optimized (1-Cycle) | 3-Stage Pipelined |
|---|---|---|---|---|
| **Slice LUTs** | 53,200 | 148 (0.28%) | 121 (0.23%) | 117 (0.22%) |
| **Slice Registers (FF)** | 106,400 | 127 (0.12%) | 115 (0.11%) | 178 (0.17%) |
| **Block RAM (RAMB18E1)** | 280 (140 tiles) | 3 (1.5 tiles - 1.07%) | 2 (1.0 tile - 0.71%) | 2 (1.0 tile - 0.71%) |
| **Bonded IOB** | 125 | 26 (20.80%) | 23 (18.40%) | 23 (18.40%) |
| **Clock Buffer (BUFG)** | 32 | 1 (3.13%) | 1 (3.13%) | 1 (3.13%) |

1. The optimized streaming architecture reduces Slice LUT utilization from 148 to 121 (-18.2%) and flip-flop count from 127 to 115 (-9.4%).
2. Eliminating the 3rd row buffer reduces on-chip BRAM consumption by 33.3%, from 3 blocks to 2 blocks.
3. In the pipelined architecture, register count increases to 178 due to inter-stage pipeline registers inserted to achieve higher operating frequencies.

## 5-4- Timing Analysis and Maximum Frequency ($F_{max}$)

Post-routing timing summary reports confirm that all three architectures achieve positive timing margins under the 100 MHz target constraint:

| Timing Parameter | Baseline Architecture | Optimized (1-Cycle) | 3-Stage Pipelined |
|---|---|---|---|
| **Target Clock Period ($T_{clk}$)** | 10.00 ns (100 MHz) | 10.00 ns (100 MHz) | 10.00 ns (100 MHz) |
| **Worst Negative Slack (WNS)** | **+3.299 ns** | **+5.302 ns** | **+5.159 ns** |
| **Total Negative Slack (TNS)** | 0.00 ns (Met) | 0.00 ns (Met) | 0.00 ns (Met) |
| **Worst Hold Slack (WHS)** | +0.083 ns | +0.166 ns | +0.143 ns |
| **Worst Pulse Width Slack (WPWS)** | +4.500 ns | +4.500 ns | +4.500 ns |
| **Minimum Period ($T_{min}$)** | **6.701 ns** | **4.698 ns** | **4.841 ns** |
| **Maximum Frequency ($F_{max}$)** | **149.23 MHz** | **212.86 MHz** | **206.57 MHz** |

### Mathematical Formulation of Maximum Operating Frequency ($F_{max}$)

The minimum stable clock period is defined as:

$$T_{min} = T_{clk} - \text{WNS}$$

The maximum operating frequency is the inverse of $T_{min}$:

$$F_{max} = \frac{1}{T_{min}} = \frac{1}{T_{clk} - \text{WNS}}$$

- **Baseline Architecture:**

$$T_{min} = 10.000 \text{ ns} - 3.299 \text{ ns} = 6.701 \text{ ns} \implies F_{max} = \frac{1}{6.701 \text{ ns}} \approx 149.23 \text{ MHz}$$

- **Optimized Architecture:**

$$T_{min} = 10.000 \text{ ns} - 5.302 \text{ ns} = 4.698 \text{ ns} \implies F_{max} = \frac{1}{4.698 \text{ ns}} \approx \mathbf{212.86 \text{ MHz}}$$

- **Pipelined Architecture:**

$$T_{min} = 10.000 \text{ ns} - 5.159 \text{ ns} = 4.841 \text{ ns} \implies F_{max} = \frac{1}{4.841 \text{ ns}} \approx 206.57 \text{ MHz}$$

## 5-5- Critical Path Analysis

Detailed path analysis in Vivado highlights key microarchitectural differences:

1. **Baseline Critical Path:**
   Originates at controller state register `u_controller/FSM_onehot_current_state_reg[5]`, propagates through sliding window logic, CSA adder stages, arithmetic multiplexers, and the CARRY4 carry-chain, terminating at destination register `u_controller/FSM_onehot_current_state_reg[6]`.
   - **Logic Levels:** 7 combinational logic levels (3x LUT6, 1x LUT3, 1x LUT2, and 2x CARRY4 primitives).
   - **Total Path Delay:** 6.670 ns (2.390 ns logic delay + 4.280 ns routing delay).

2. **Optimized Architecture Critical Path:**
   Heavy combinational arithmetic chains are eliminated; the critical path is confined to controller signal propagation from `u_controller/col_reg[1]` to the internal counter register.
   - **Logic Levels:** Only 3 logic levels (2x LUT5 and 1x LUT6).
   - **Total Path Delay:** 4.439 ns (1.050 ns logic delay + 3.389 ns routing delay).

# 5-6- Vivado Implementation Screenshots

## 1. Initial Unconstrained Implementation and 176W Power Anomaly (Files 01.png, 02.png, 03.png)

These screenshots illustrate the initial implementation phase without clock constraints and prior to BRAM inference, resulting in 176W power dissipation and 6,402 LUTs:

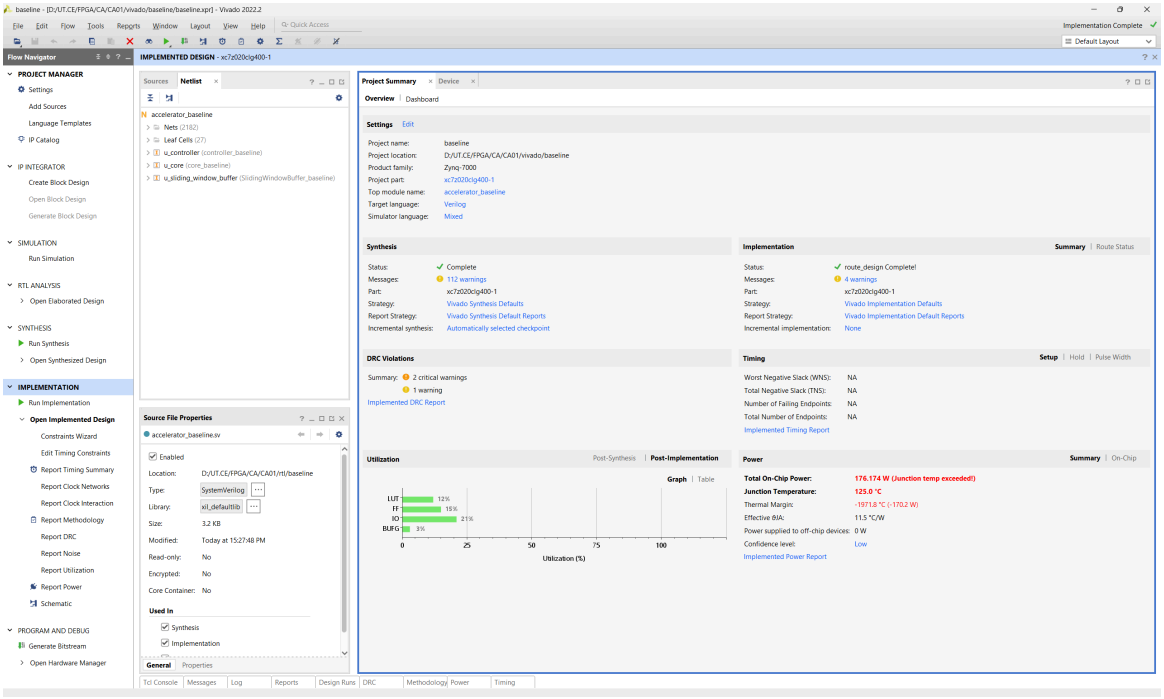



Figure 10: Project Summary of Unconstrained Implementation: 176.174 W Power and 'Junction temp exceeded' Alarm (01.png)

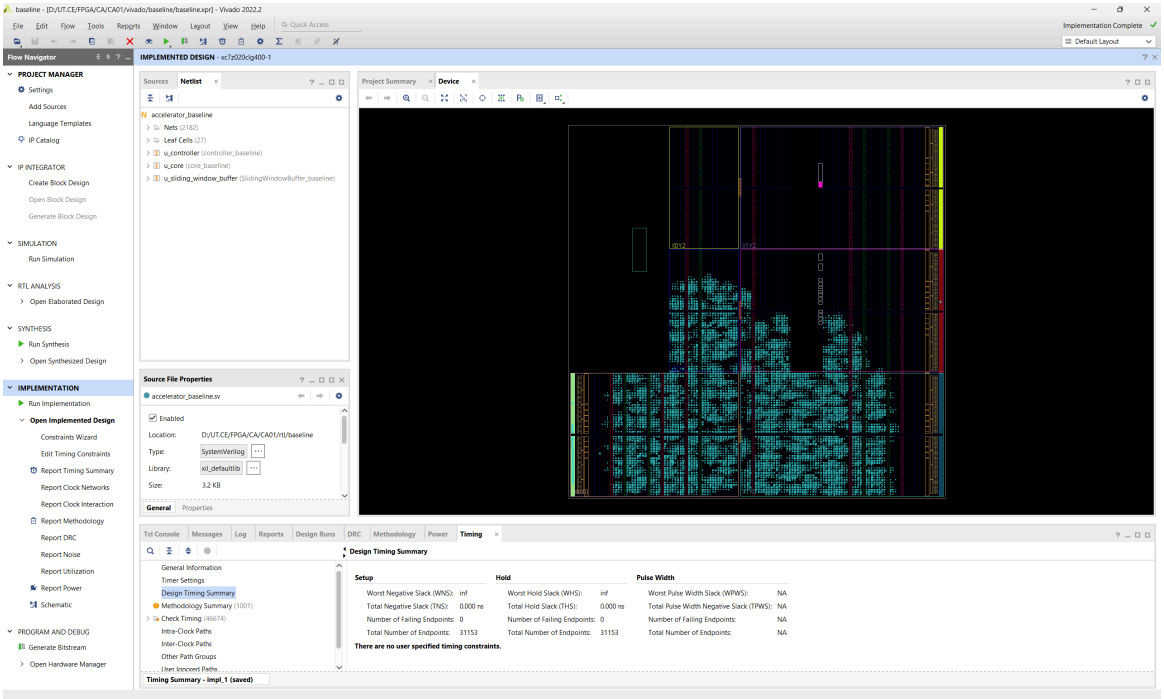



Figure 11: Device Floorplan and Timing Report with 'There are no user specified timing constraints' Warning (02.png)

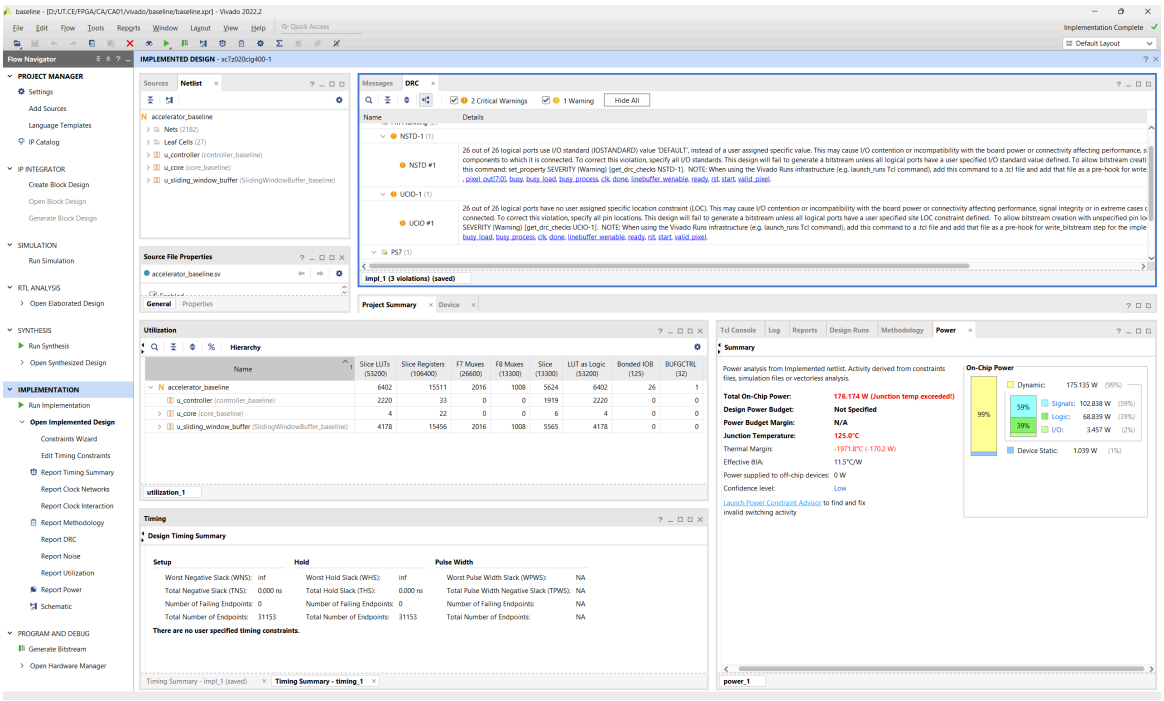



Figure 12: Detailed Power Report (176.174 W) and Resource Utilization Showing 6,402 LUTs and 15,511 FFs (03.png)

## 2. Intermediate Stage: BRAM Inference Without Clock Constraints (Files with 'true' Suffix)

Restructuring the line buffer code allowed automatic inference of dedicated `RAMB18E1` primitives, reducing LUT utilization to 149 and lowering power by 95% to 8.1W:

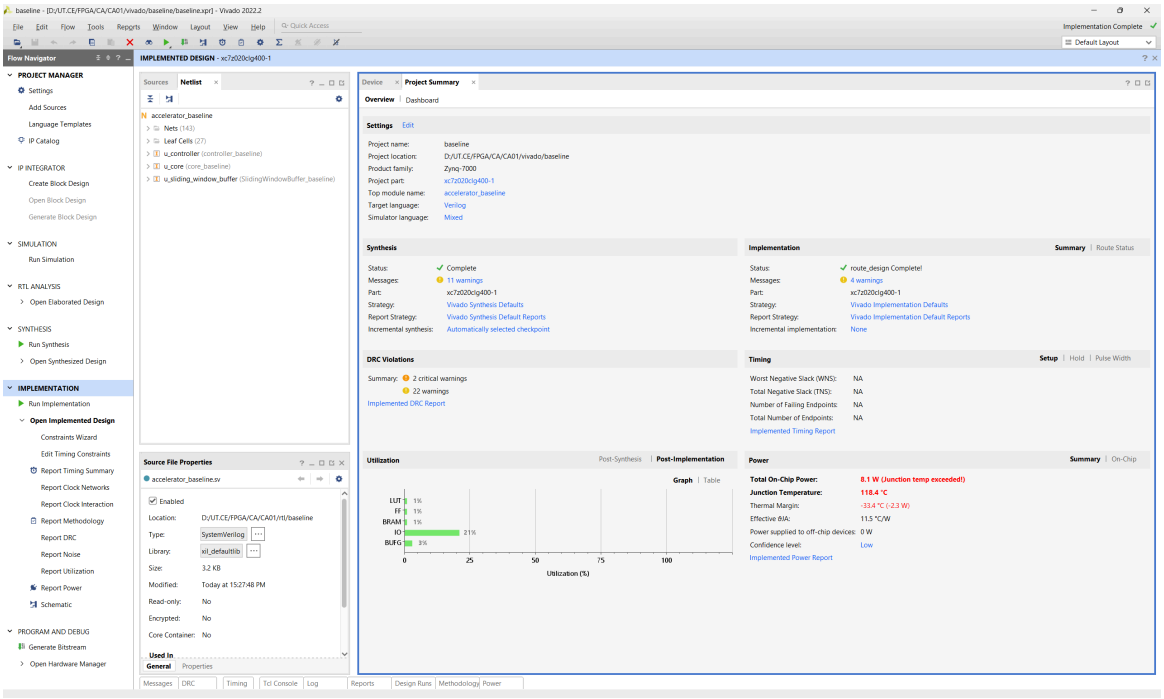



Figure 13: Project Summary After BRAM Inference: Power Reduced to 8.1 W in Unconstrained Mode (01-true.png)

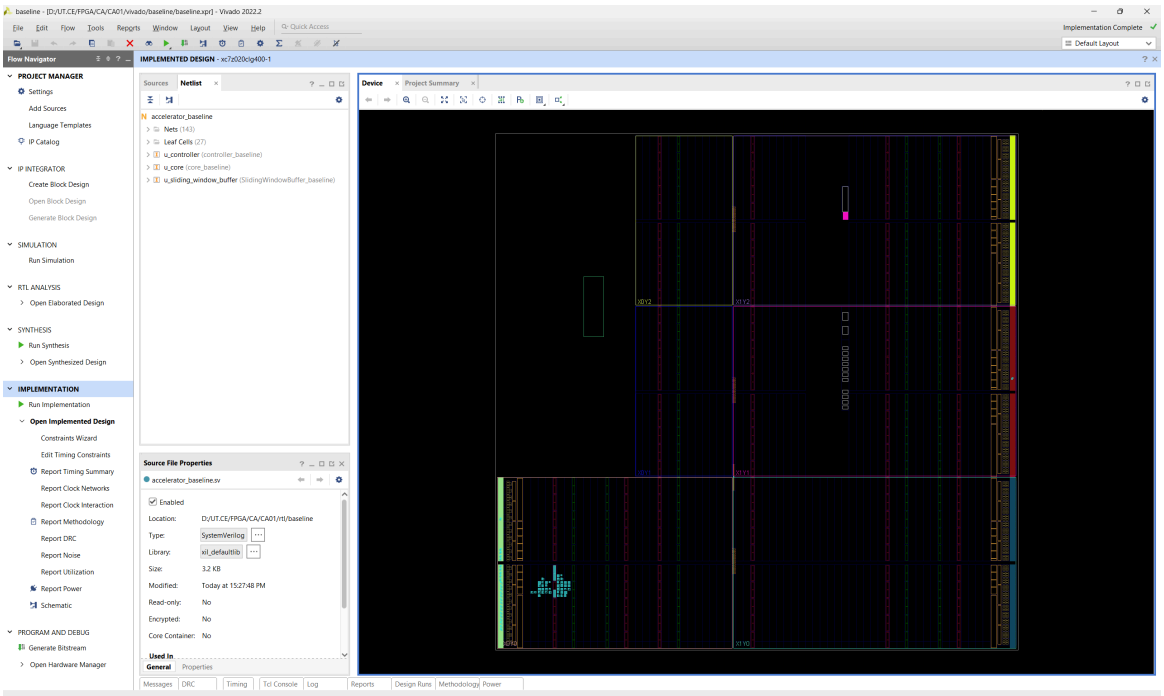


Figure 14: Device Floorplan Showing Successful Placement of Dedicated BRAM Primitives (02-true.png)

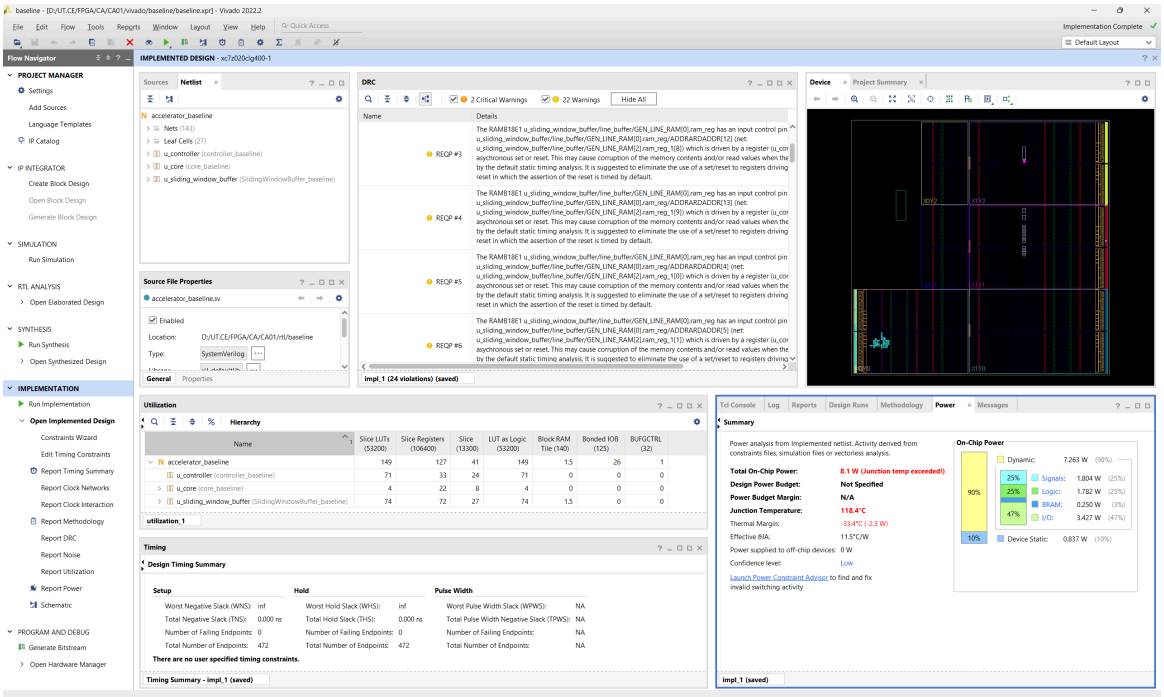


Figure 15: Detailed Resource Table After BRAM Inference: 149 LUTs, 127 FFs, and 1.5 BRAM Tiles (03-true.png)

## 3. Final Constrained Implementation at 100 MHz

With the 10.0 ns clock constraint applied via XDC, timing was fully closed, reducing total power to 117 mW and junction temperature to 26.3 °C:

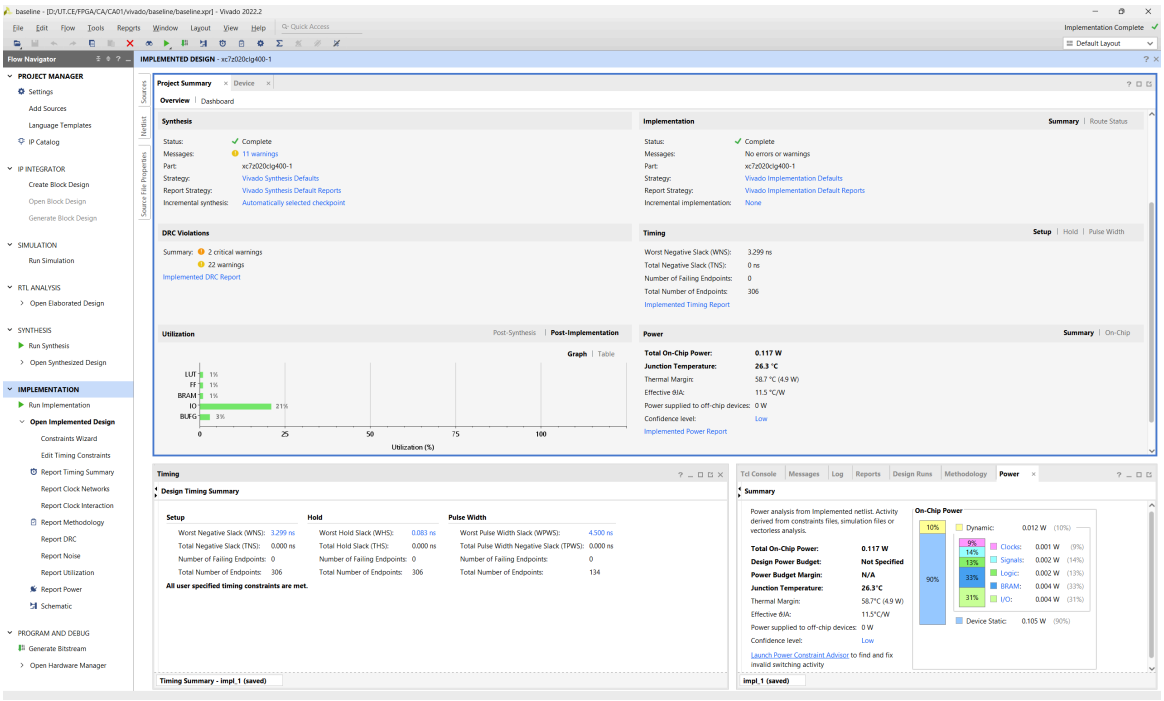



Figure 16: Final Implementation Summary for Baseline Architecture with 100 MHz Constraint (WNS = +3.299 ns, Power = 0.117 W)

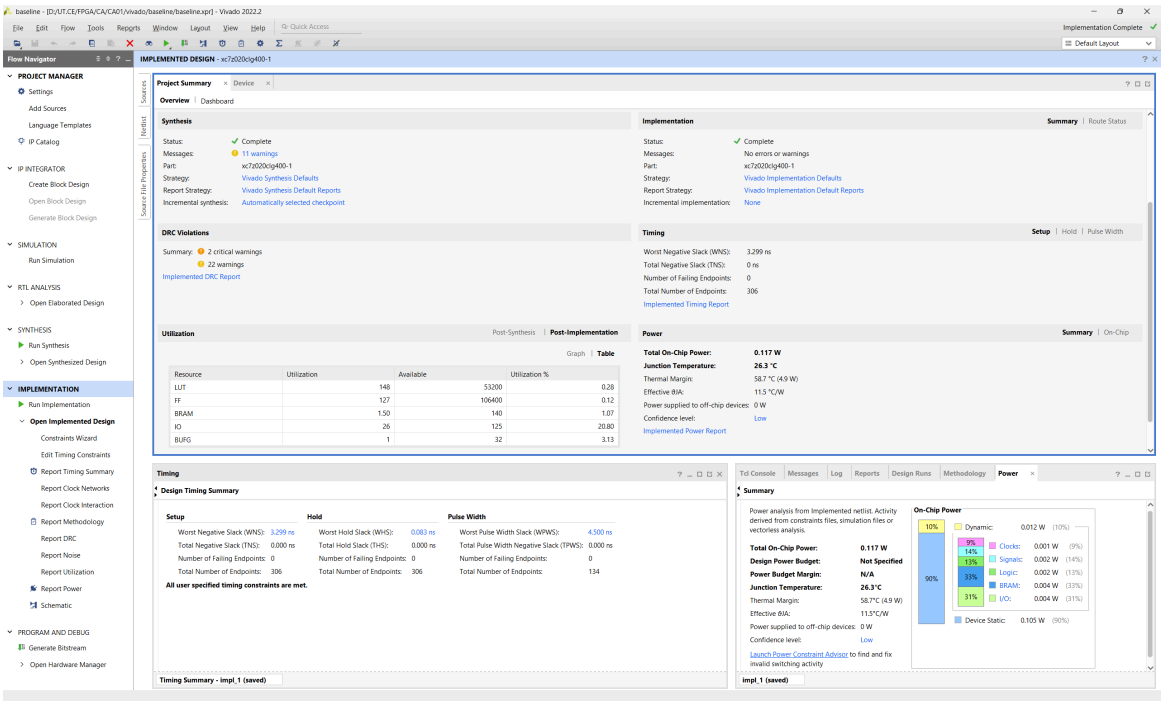



Figure 17: Post-Implementation Utilization Table for the Baseline Architecture

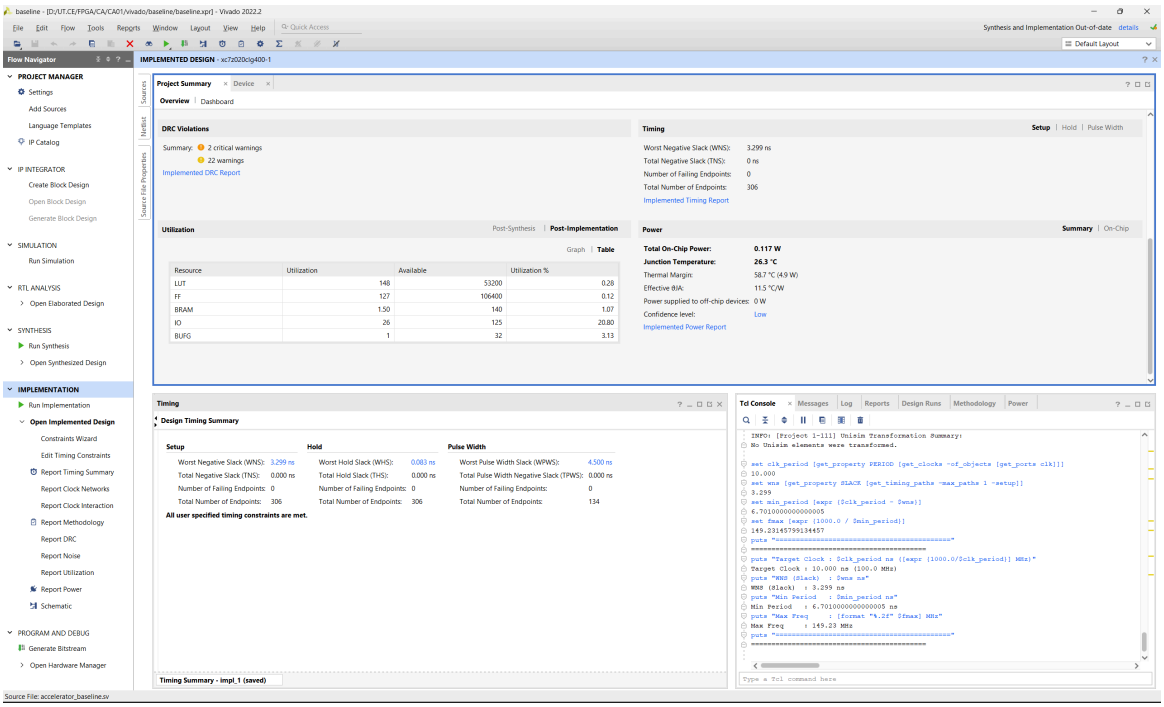



Figure 18: Automated Tcl Calculation of Maximum Operating Frequency (149.23 MHz) for Baseline Architecture

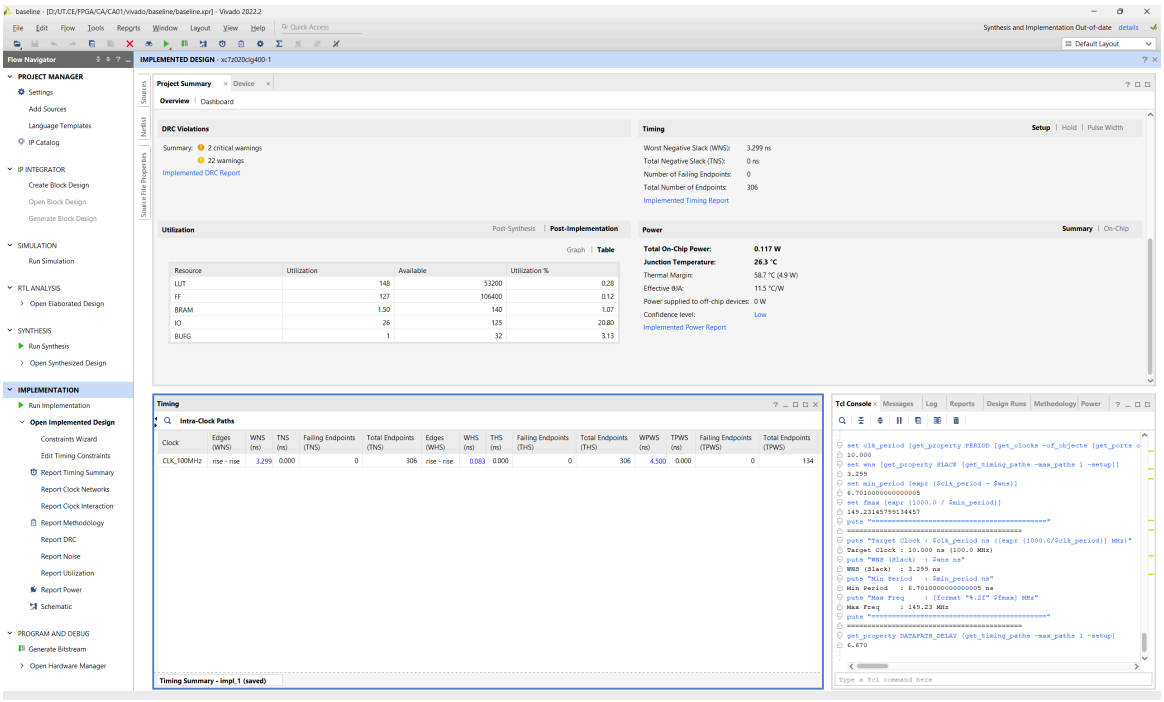



Figure 19: Detailed Intra-Clock Timing Paths and Slack Analysis for Baseline Architecture

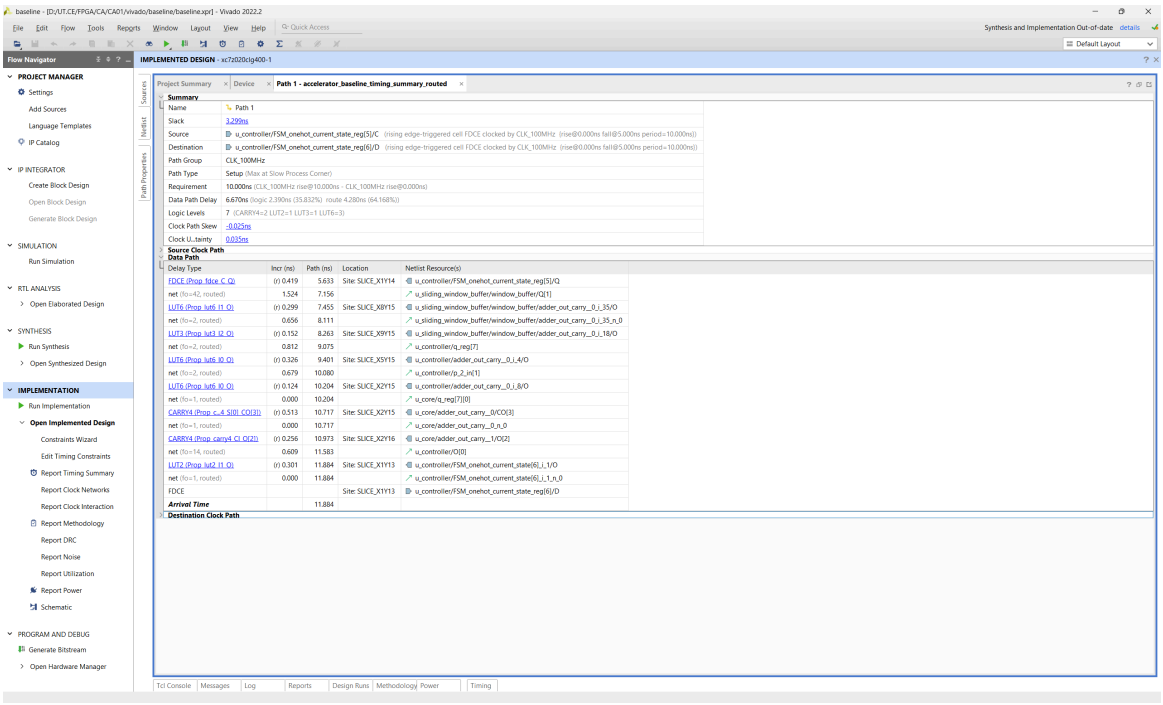



Figure 20: Critical Path Breakdown with 7 Logic Levels and 6.670 ns Total Delay in Baseline Architecture

## 4. Implementation Results for Optimized Streaming Architecture

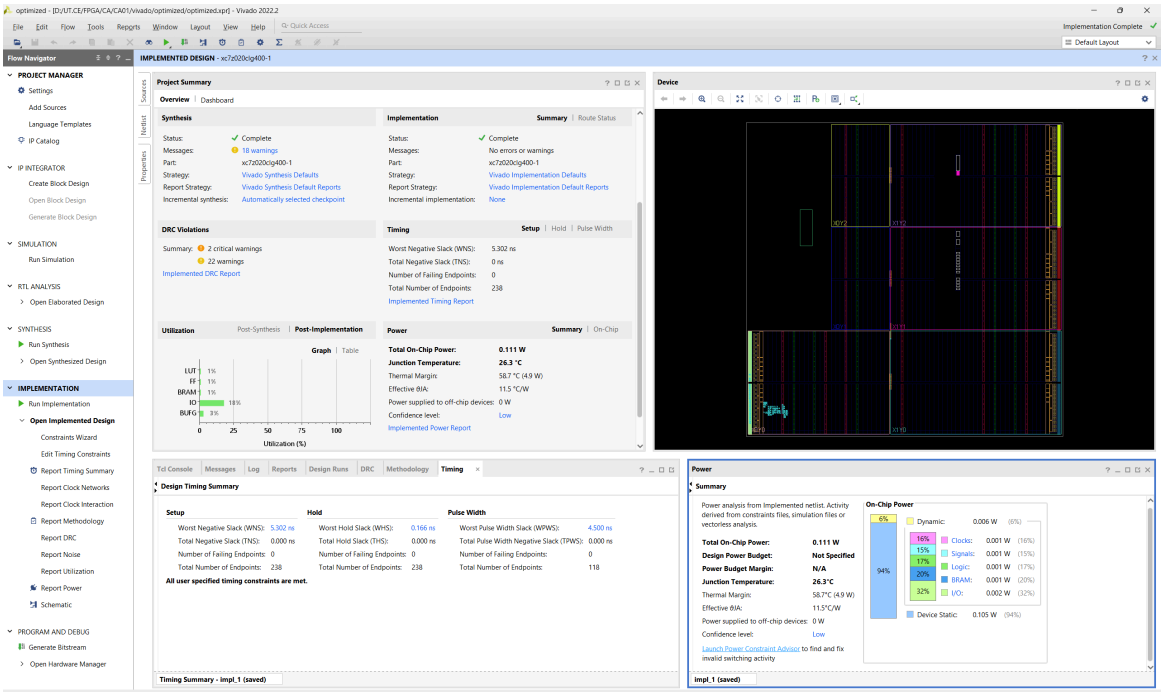



Figure 21: Project Summary for Optimized Architecture Showing Power Breakdown and Setup Margin (WNS = +5.302 ns)

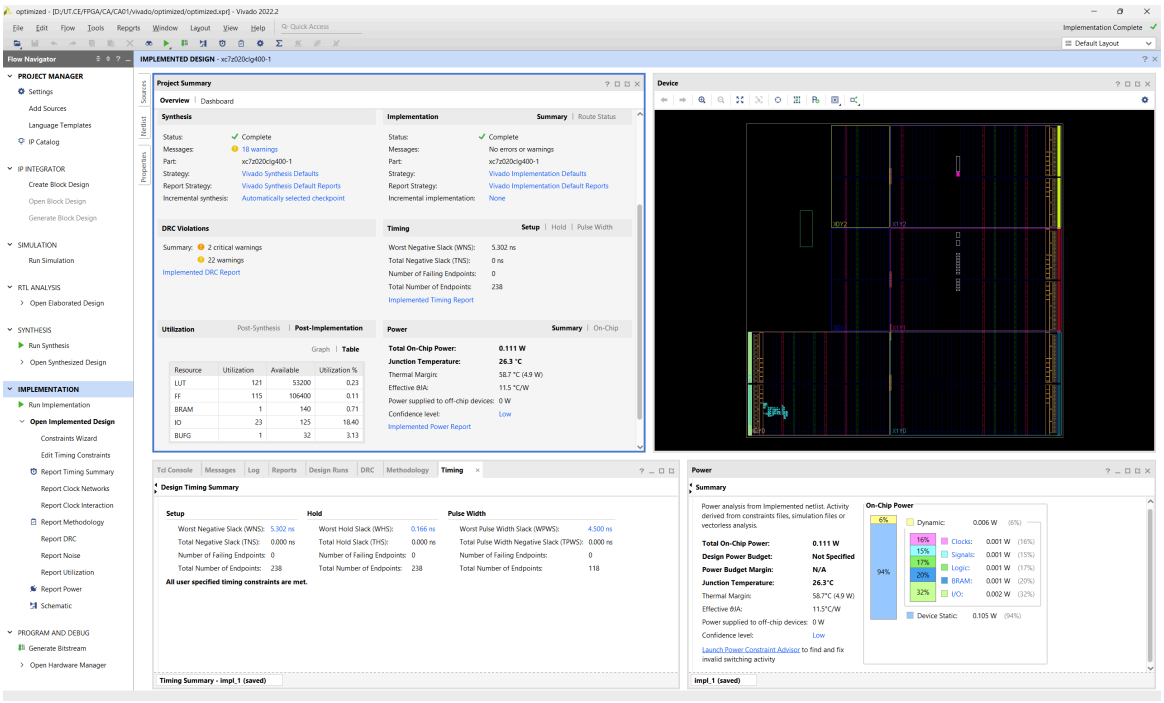



Figure 22: Resource Utilization Table for Optimized Architecture Demonstrating BRAM Reduction to 1 Block

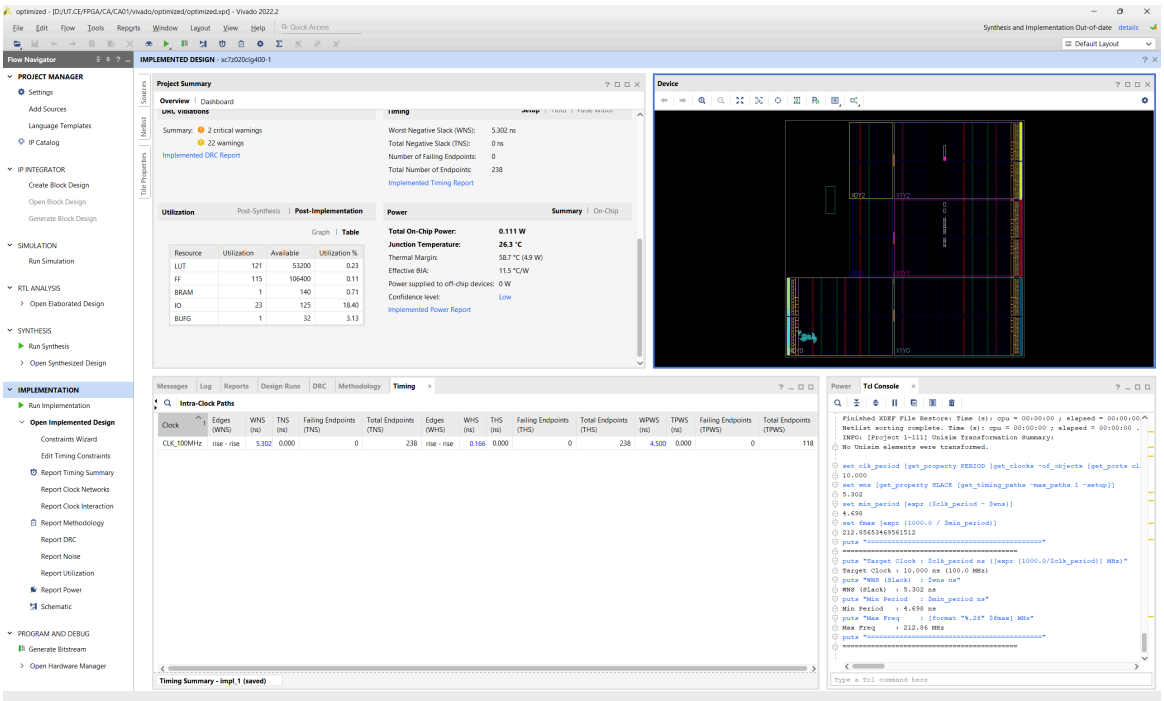



Figure 23: Tcl Console Computation of Maximum Operating Frequency (212.86 MHz) for Optimized Architecture

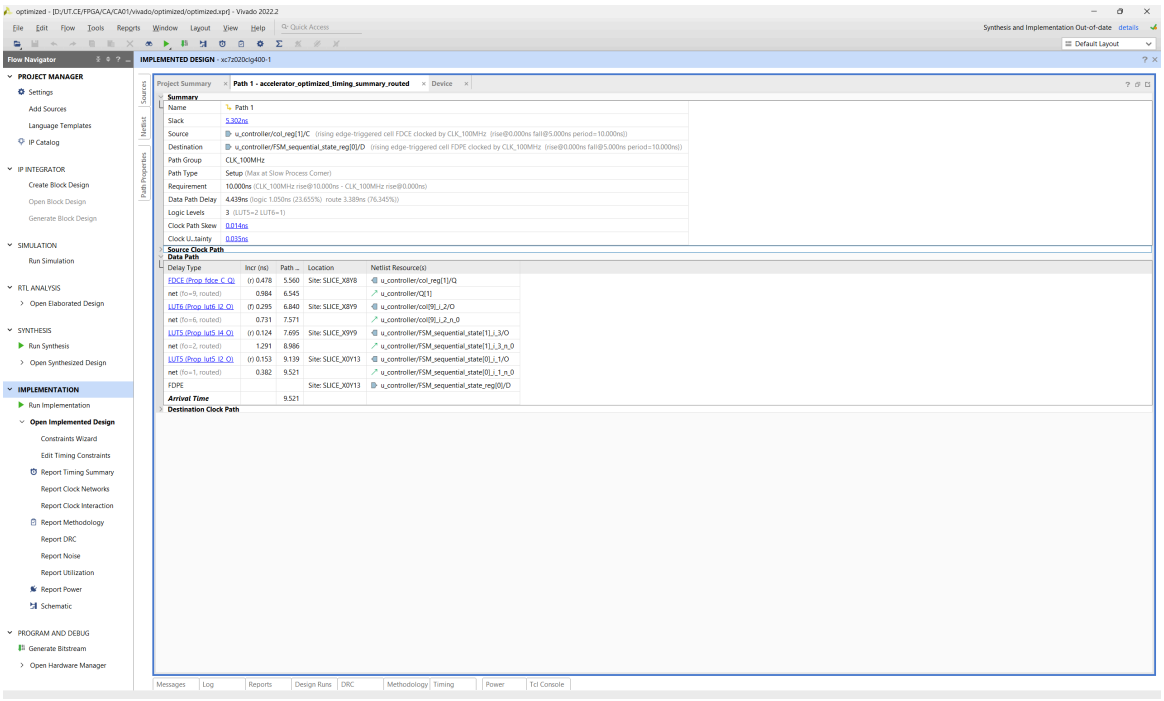



Figure 24: Critical Path Analysis for Optimized Architecture: Logic Levels Reduced to 3 with 4.439 ns Delay

## 5. Implementation Results and Device Floorplan for Pipelined Architecture

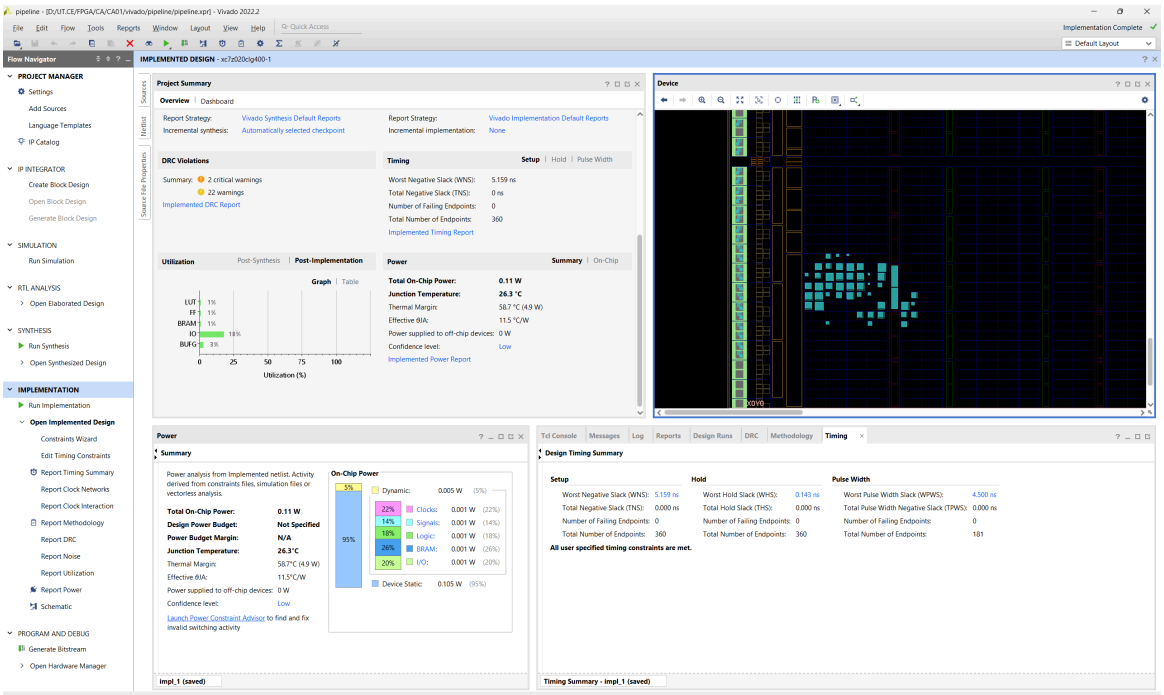



Figure 25: Implementation Summary for 3-Stage Pipelined Architecture (WNS = +5.159 ns)

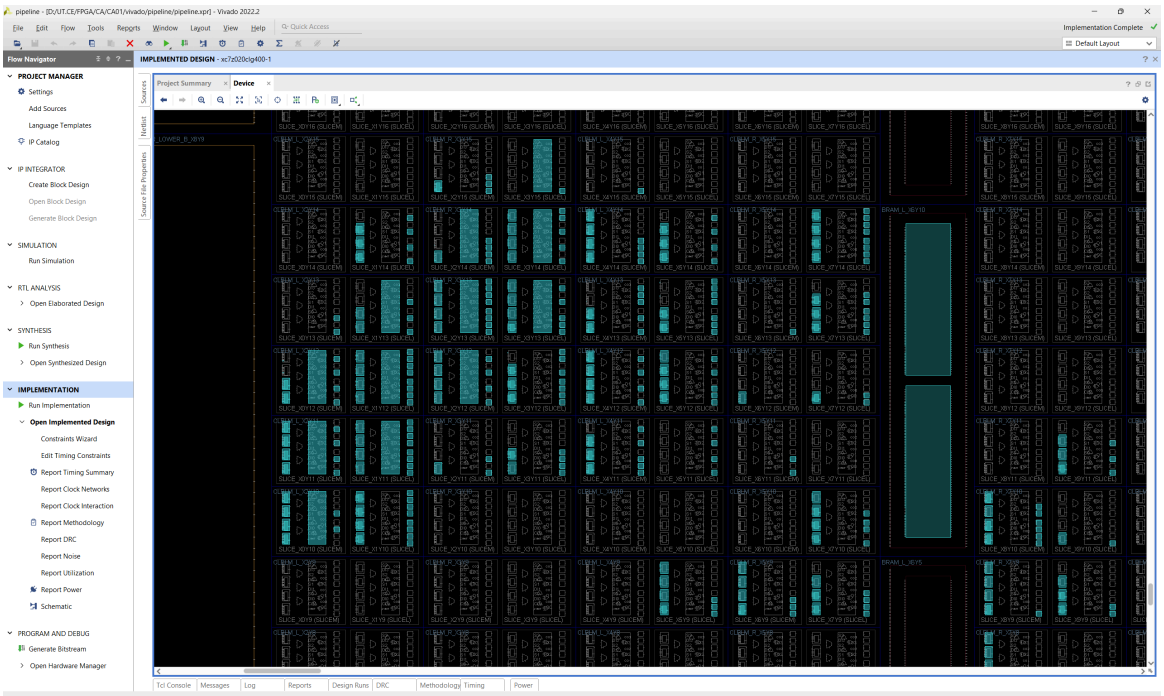


Figure 26: Device Floorplan Illustrating Logic Slice and BRAM Allocation for Pipelined Architecture

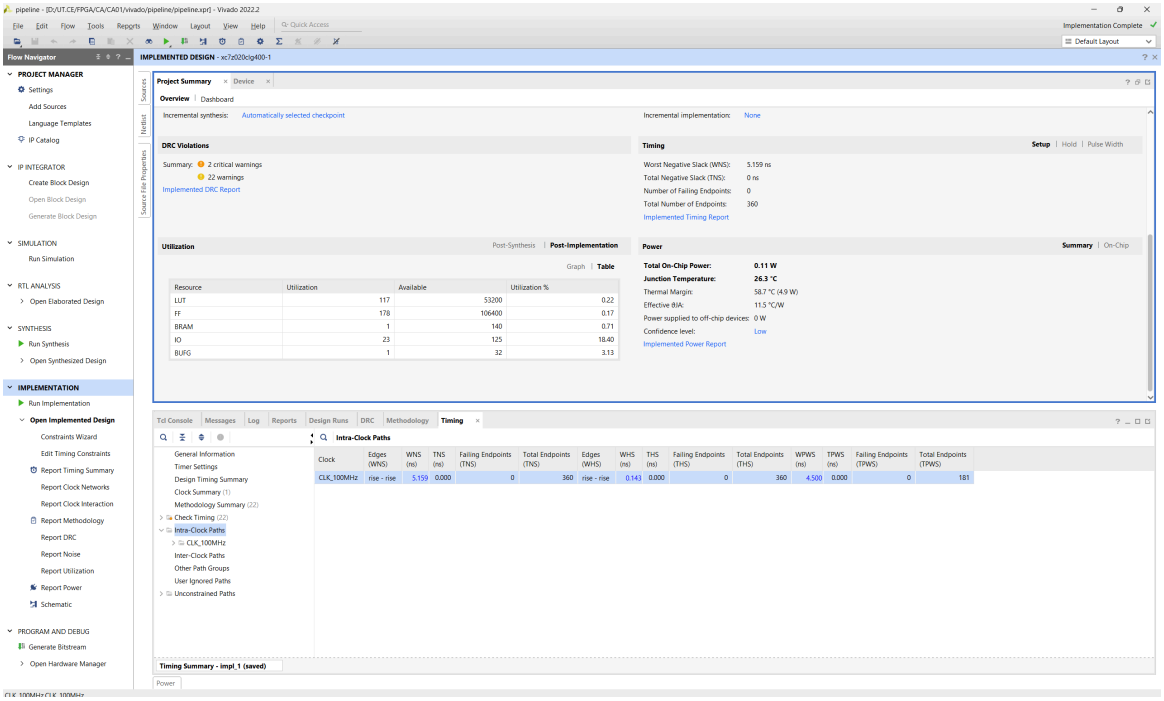


Figure 27: Intra-Clock Timing Path Analysis for Pipelined Architecture

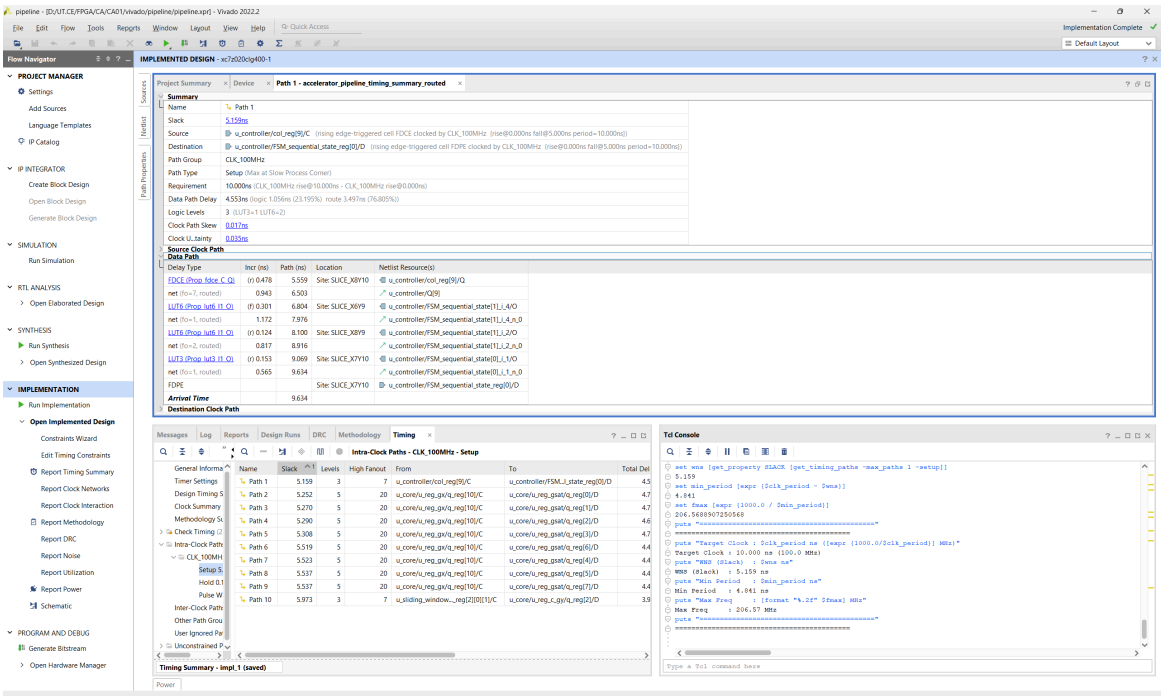



Figure 28: Critical Path Report and Frequency Analysis (206.57 MHz) for Pipelined Architecture